JSPM's
Rajarshi Shahu College of Engineering, Tathwade Pune-33
(An Autonomous Institute Affiliated to Savitribai Phule Pune University, Pune)

Department of Computer Engineering

Lab Manual

of

CS3201L

Machine Learning Laboratory

Contents

Vision 3
Mission 3
PEOs, PSOs 4
Program Outcomes 5
Course Outcomes and its mapping with POs and PSOs 6
Lab Assignment : 1 8
Lab Assignment: 2 12
Lab Assignment: 3 19
Lab Assignment: 4 25
Lab Assignment: 5 30
Lab Assignment: 6 35
Lab Assignment: 7 41
Lab Assignment: 8 46
Lab Assignment: 9 ..... **52**
Lab Assignment: 10 ..... **57**

Vision and Mission of Institute

Vision of Institute

To be a recognized institution for academic excellence, innovation, research, leadership and holistic development for socially responsible professionals driving sustainable national and global progress.

Mission of Institute

M1. To deliver transdisciplinary, outcome based and experiential education that nurtures academic excellence, critical thinking, professional competence and life long learning.

M2. To promote research, innovation, entrepreneurship and collaborative knowledge creation to address national and global challenges.

M3. To instil ethical values, leadership, social responsibility and global competence for holistic and sustainable human development.

Vision and Mission of Department

Vision of Department

To become a driving force in Computer Engineering through academic excellence, cutting-edge innovation, and technological advancements for sustainable socio-economic growth.

Mission of Department

M1: To equip students with strong foundations in fundamental computer engineering and emerging technologies through an inclusive environment to solve complex real world problem.

M2: To promote a culture of research, innovation, and entrepreneurship for developing successful computer professionals.

M3: To inculcate ethical values, leadership qualities, teamwork, and social responsibility among students for lifelong learning and sustainable societal development.

Program Educational Objectives (PEOs)

PEO I: Professional Competence and Career Development:

Graduates will establish successful careers or pursue higher education by applying Computer Engineering knowledge, technical skills, and professional competence to solve real-world problems.

PEO II: Research, Innovation, and Lifelong Learning:

Graduates will engage in research, innovation, entrepreneurship, and lifelong learning to address emerging technological challenges and contribute to industrial and societal advancement.

PEO III: Ethics, Leadership, and Social Responsibility:

Graduates will contribute to sustainable socio economic development by applying ethical values, effective teamwork and leadership within their professional core operational framework.

Program Specific Outcomes (PSOs)

A graduate of the Computer Engineering Program will demonstrate

PSO1: Software Systems and Computing:

Graduates will be able to analyze, design, develop, and evaluate efficient software and computing systems by applying Computer Engineering fundamentals, software engineering principles, and modern development tools.

PSO2: Intelligent and Emerging Technologies:

Graduates will be able to develop secure, scalable, and intelligent solutions using emerging technologies such as Artificial Intelligence, Data Science, Cloud Computing, Cyber Security and related emerging technologies.

PSO3: Innovation, Research, and Entrepreneurship:

Graduates will be able to apply research methodologies, emerging technologies, and entrepreneurial skills to design, develop, and validate innovative computing solutions for complex engineering problems.

Knowledge and Attitude Profile (WK)

WK1: A systematic, theory-based understanding of the natural sciences applicable to the discipline and awareness of relevant social sciences.

WK2: Conceptually-based mathematics, numerical analysis, data analysis, statistics and formal aspects of computer and information science to support detailed analysis and modelling applicable to the discipline.

WK3: A systematic, theory-based formulation of engineering fundamentals required in the engineering discipline.

WK4: Engineering specialist knowledge that provides theoretical frameworks and bodies of knowledge for the accepted practice areas in the engineering discipline; much is at the forefront of the discipline.

WK5: Knowledge, including efficient resource use, environmental impacts, whole-life cost, reuse of resources, net zero carbon, and similar concepts, that supports engineering design and operations in a practice area.

WK6: Knowledge of engineering practice (technology) in the practice areas in the engineering discipline.

WK7: Knowledge of the role of engineering in society and identified issues in engineering practice in the discipline, such as the professional responsibility of an engineer to public safety and sustainable development.

WK8: Engagement with selected knowledge in the current research literature of the discipline, awareness of the power of critical thinking and creative approaches to evaluate emerging issues.

WK9: Ethics, inclusive behavior and conduct. Knowledge of professional ethics, responsibilities, and norms of engineering practice. Awareness of the need for diversity by reason of ethnicity, gender, age, physical ability etc. with mutual understanding and respect, and of inclusive attitudes.

Program Outcomes(POs)

Engineering Graduates will be able to:

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Course Outcomes and its mapping with Pos and PSOs

CO1	Apply data cleaning, transformation, and feature extraction techniques	BL3
CO2	Implement machine learning algorithms for classification, regression, and clustering.	BL3
CO3	Analyze model performance using metrics like accuracy, precision, recall, and AUC.	BL4
CO4	Design Machine Learning solutions for real-world applications using appropriate algorithms and techniques.	BL6

CO – PO Mapping:

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
CO1	3	2	1	2	3	-	-	-	-	1	-	2
CO2	3	3	2	2	3	-	-	-	-	1	-	2
CO3	3	3	2	3	3	-	-	-	-	1	-	2
CO4	3	3	3	3	3	2	1	1	2	2	1	3

CO – PSO Mapping:

PSO	**PSO1**	**PSO2**	**PSO3**
CO1	3	2	1
CO2	3	3	2
CO3	3	3	2
CO4	3	3	3

ASSIGNEMENT NO. 1

Implement Simple Linear Regression using a suitable dataset

AIM: Implement Simple Linear Regression using a suitable dataset. Comment on the results obtained from each technique.

OBJECTIVES:

- To understand the fundamentals of Linear regression.
- To implement Linear in Python using scikit-learn.
- To analyze the performance model.
- To visualize the models and interpret the results.

PROBLEM STATEMENT:

Develop a program to implement Simple Linear Regression using a suitable dataset. Evaluate and compare the performance of these models using appropriate datasets.

SOFTWARE REQUIRED:

- **Python**
- **Libraries: scikit-learn, numpy, matplotlib, pandas**

INPUT: Dataset

OUTPUT:

- Trained regression models.
- Performance metrics: Mean Squared Error (MSE), coefficient of determination R^2.
- Graphs and plots visualizing predictions and decision boundaries.
- Observations and comparison of model performances.

THEORY:

Regression

The term regression is used when you try to find the relationship between variables.

In Machine Learning, and in statistical modeling, that relationship is used to predict the outcome of future events.

Linear Regression

Linear regression is a type of supervised machine learning algorithm that learns from labeled datasets and maps the data points using an optimized linear function, which can then be used for making predictions on new data. It assumes a linear relationship between the input and output, meaning the output changes at a constant rate as the input changes. This relationship is represented by a straight line.

For instance, consider the task of predicting a student's exam score based on the number of hours studied. As the number of study hours increases, exam scores tend to increase as well. In this case:

- **Independent variable (input):** Hours studied, as it is the factor being controlled or observed.
- **Dependent variable (output):** Exam score, as it varies depending on the number of hours studied.

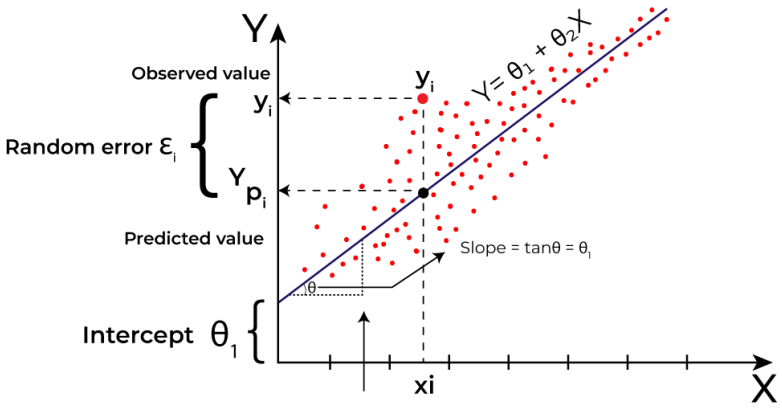
The independent variable is used to predict the dependent variable in the linear regression model.

For simple linear regression (with one independent variable), the best-fit line is represented by the equation $y=mx+b$

Where:

- **y** is the predicted value (dependent variable)
- **x** is the input (independent variable)
- **m** is the slope of the line (how much y changes when x changes)
- **b** is the intercept (the value of y when x = 0)

The best-fit line will be the one that optimizes the values of m (slope) and b (intercept) so that the predicted y values are as close as possible to the actual data points.



Conclusion:

- Linear Regression is best for continuous output with linear relationships
- Simple and fast, but not suitable for non-linear patterns

Python Code Implementation

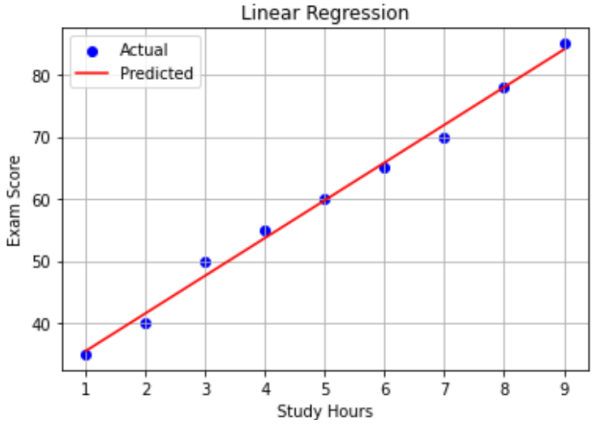
LINEAR REGRESSION

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
```

```
# Dataset: Study Hours vs Exam Scores
X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9]).reshape(-1, 1)
y = np.array([35, 40, 50, 55, 60, 65, 70, 78, 85])

# Model training
model = LinearRegression()
model.fit(X, y)
y_pred = model.predict(X)

# Visualization
plt.scatter(X, y, color='blue', label='Actual')
plt.plot(X, y_pred, color='red', label='Predicted')
plt.title('Linear Regression')
plt.xlabel('Study Hours')
plt.ylabel('Exam Score')
plt.legend()
plt.grid(True)
plt.show()
```




```
# Evaluation
print("Linear Regression R² Score:", r2_score(y, y_pred))
```

Linear Regression R² Score: 0.9931241255246852

ASSIGNEMENT NO. 2

Implement Multiple Linear Regression and Polynomial Regression using a suitable dataset.

AIM: Implement **Multiple Linear Regression and Polynomial Regression** using suitable datasets and compare the performance of the models using appropriate evaluation metrics.

OBJECTIVES:

- To understand the fundamentals of **Multiple Linear Regression** and **Polynomial Regression**.
- To implement Multiple Linear Regression and Polynomial Regression in Python using **scikit-learn**.
- To understand the effect of multiple independent variables on a continuous dependent variable.
- To understand how polynomial features can model **non-linear relationships**.
- To evaluate and compare the performance of the models using appropriate regression metrics.
- To visualize and interpret the prediction results.

PROBLEM STATEMENT:

Develop a Python program to implement **Multiple Linear Regression and Polynomial Regression** using suitable datasets. Train the models, predict the output for test data, evaluate their performance using appropriate regression metrics, and compare the obtained results.

SOFTWARE REQUIRED:

- **Python**
- **Libraries: scikit-learn, numpy, matplotlib, pandas**

INPUT:

Suitable dataset containing:

- Multiple independent variables and one continuous dependent variable for **Multiple Linear Regression.**
- An independent variable and a continuous dependent variable having a non-linear relationship for **Polynomial Regression**.

OUTPUT:

- Trained **Multiple Linear Regression model**.
- Trained **Polynomial Regression model**.
- Predicted values for the test data.
- Performance metrics:
    - **Mean Squared Error (MSE)**

- **Coefficient of Determination (R^2)**

- Graphs showing actual and predicted values.
- Comparison and interpretation of model performance

THEORY:

1. Multiple Linear Regression

Multiple Linear Regression is a supervised machine learning algorithm used to predict a **continuous dependent variable using two or more independent variables**.

The general equation is:

$$y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_n X_n$$

Where:

- y is the dependent variable
- $X_1, X_2, \cdots X_n$ are the independent variables
- β_0 is the intercept
- $\beta_1, \beta_2, \cdots \beta_n$ are the slopes

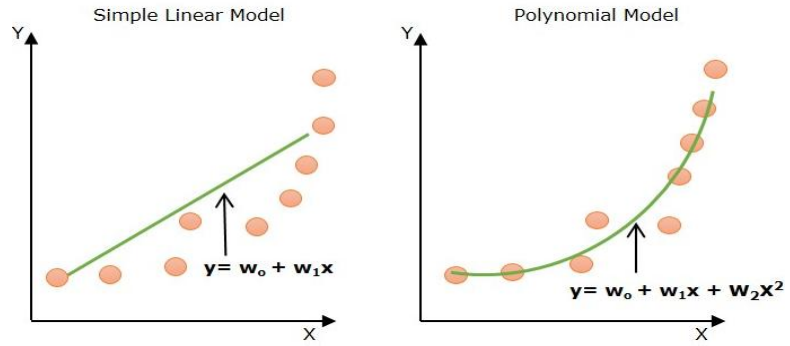
The goal of the algorithm is to find the best fit line equation that can predict the values based on the independent variables. A regression model learns from the dataset with known X and y values and uses it to predict y values for unknown X.

2. Polynomial regression

Polynomial Linear Regression is a type of regression analysis in which the relationship between the independent variable and the dependent variable is modeled as an n-th degree polynomial function. Polynomial regression allows for a more complex relationship between the variables to be captured beyond the linear relationship in simple linear regression and multiple linear regression.

In machine learning (ML) and data science, choosing between a linear regression or polynomial regression depends upon the characteristics of the dataset. A non-linear dataset can't be fitted with a linear regression. If we apply linear regression to a nonlinear dataset, it will not be able to capture the non-linear patterns in the data.

Look at the below diagram to understand why we need polynomial regression for non-linear data.



The above diagram shows the simple linear model hardly fits the data points whereas the polynomial model fits most of the data points.

Equation of Polynomial Regression Model

In machine learning, the general formula for polynomial regression of degree n is as follows

$y=w0+w1x+w_2x_2+w_3x_3+\ldots+w_nx_n+\epsilon$

Where

- y is the dependent variable (output).
- x is the independent variable (input).
- w0,w1,w2,…,wn are the coefficients (parameters) of the model.
- n is the degree of the polynomial (the highest power of x).
- ϵ is the error term or residual, representing the difference between the observed value and the model's prediction.

For a quadratic (second-degree) polynomial regression, the formula would be:

$y=w0+w1x+w_2x_2+\epsilon$

This would fit a parabolic curve to the data points.

Conclusion

1. Multiple Linear Regression effectively predicts a continuous output using multiple independent variables and performs well when the relationships between variables are approximately linear.
2. Polynomial Regression can model non-linear relationships by introducing higher-order polynomial terms and is useful for fitting curved patterns in the data.

POLYNOMIAL REGRESSION

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
```

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

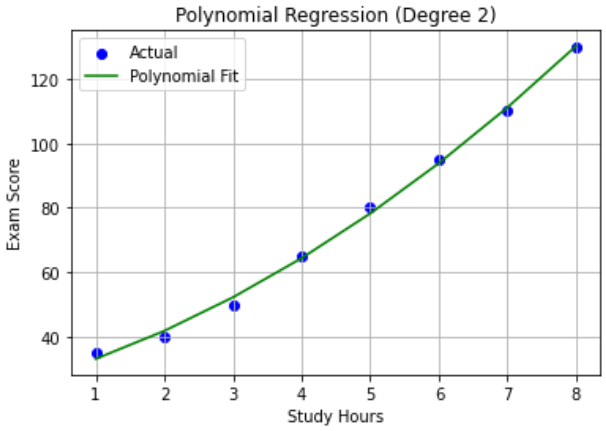
# Dataset: Study Hours and Exam Scores
X = np.array([1, 2, 3, 4, 5, 6, 7, 8]).reshape(-1, 1)
y = np.array([35, 40, 50, 65, 80, 95, 110, 130])

# Transform to polynomial features
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)

# Model training
poly_model = LinearRegression()
poly_model.fit(X_poly, y)

# Prediction
y_poly_pred = poly_model.predict(X_poly)

# Visualization
plt.scatter(X, y, color='blue', label='Actual')
plt.plot(X, y_poly_pred, color='green', label='Polynomial Fit')
plt.title('Polynomial Regression (Degree 2)')
plt.xlabel('Study Hours')
plt.ylabel('Exam Score')
plt.legend()
plt.grid(True)
plt.show()
print("Polynomial Regression R² Score:",  r2_score(y, y_poly_pred))
```



Polynomial Regression R^2 Score: 0.9978099943892418

2. Multiple Regression:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
X = np.array([
    [2, 60, 50],
    [3, 65, 55],
    [4, 70, 60],
    [5, 75, 65],
    [6, 80, 75],
    [7, 85, 80],
    [8, 90, 85],
    [9, 95, 90]
])

y = np.array([40, 45, 50, 60, 65, 75, 85, 90])

model = LinearRegression()
model.fit(X, y)
y_pred = model.predict(X)
print("Intercept:", model.intercept_)
print("Coefficients:", model.coef_)

new_student = np.array([[6, 80, 75]])
predicted_score = model.predict(new_student)
print("Predicted Exam Score:", predicted_score[0])
plt.scatter(y, y_pred, color='blue', label='Predicted Scores')
plt.plot([min(y), max(y)], [min(y), max(y)],
         color='green', label='Perfect Prediction')
plt.title('Multiple Linear Regression')
plt.xlabel('Actual Exam Score')
plt.ylabel('Predicted Exam Score')
plt.legend()
```

```
plt.grid(True)
plt.show()

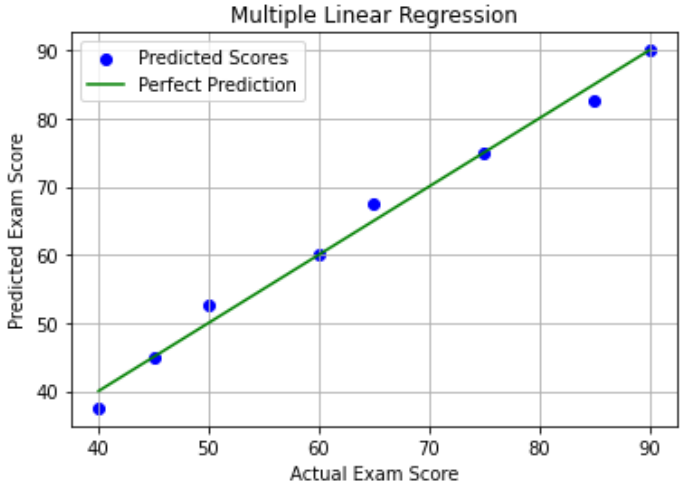
# Evaluation
print("Multiple Regression R² Score:", r2_score(y, y_pred))
```

Output:

Intercept: -49.61538461538454

Coefficients: [2.88461538e-01 1.44230769e+00 4.24396588e-16]

Predicted Exam Score: 67.49999999999999



Multiple Regression R² Score: 0.9895287958115183

ASSIGNEMENT NO. 3

Implement Logistic Regression using a suitable dataset.

AIM: Implement Logistic Regression using a suitable classification dataset and evaluate the performance of the model using appropriate classification evaluation metrics.

OBJECTIVES:

- To understand the fundamentals of Logistic Regression.
- To implement Logistic Regression in Python using scikit-learn.
- To understand the use of the Sigmoid function for classification.
- To predict the probability and class of a categorical target variable.
- To evaluate the performance of the classification model using appropriate metrics.
- To visualize and interpret the classification results.

PROBLEM STATEMENT:

Develop a Python program to implement Logistic Regression using a suitable dataset. Train the model using the training data, predict the class labels and probabilities for the test data, evaluate the model using appropriate classification metrics, and interpret the obtained results.

SOFTWARE REQUIRED:

- Python
- Libraries: scikit-learn, numpy, matplotlib, pandas

INPUT:

A suitable dataset containing:

- One or more independent/input variables (features).
- One categorical dependent/output variable (target).

OUTPUT:

- Trained Logistic Regression model.
- Predicted class labels for the test data.
- Predicted probabilities for each class.
- Confusion Matrix.
- Classification performance metrics:
    - Accuracy
    - Precision
    - Recall
    - F1-score
    - Graphs/plots visualizing the classification results.

- Observations and interpretation of model performance

THEORY:

Logistic Regression is a **supervised machine learning algorithm used primarily for classification problems**.

Although its name contains the word "Regression", it is generally used to predict the **probability of a categorical outcome**.

For example, Logistic Regression can be used to predict:

- Spam / Not Spam
- Pass / Fail
- Disease / No Disease
- Yes / No
- Customer will buy / will not buy

The nature of target or dependent variable is dichotomous, which means there would be only two possible classes. In simple words, the dependent variable is binary in nature having data coded as either 1 (stands for success/yes) or 0 (stands for failure/no).

Mathematically, a logistic regression model predicts P(Y=1) as a function of X. It is one of the simplest ML algorithms that can be used for various classification problems such as spam detection, Diabetes prediction, cancer detection etc.

Logistic regression can be divided into following types −

Binary or Binomial

In such a kind of classification, a dependent variable will have only two possible types either 1 and 0. For example, these variables may represent success or failure, yes or no, win or loss etc.

Multinomial

In such a kind of classification, dependent variable can have 3 or more possible unordered types or the types having no quantitative significance. For example, these variables may represent "Type A" or "Type B" or "Type C".

Ordinal

In such a kind of classification, dependent variable can have 3 or more possible ordered types or the types having a quantitative significance. For example, these variables may represent "poor" or "good", "very good", "Excellent" and each category can have the scores like 0,1,2,3.

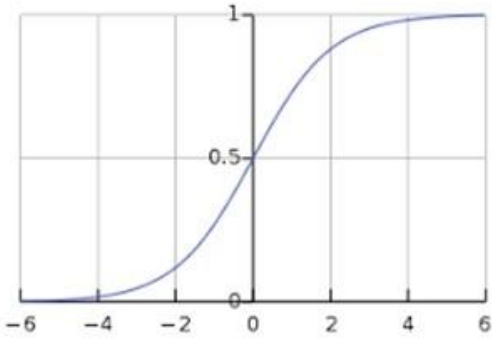
The simplest form of logistic regression is binary or binomial logistic regression in which the target or dependent variable can have only 2 possible types either 1 or 0. It allows us to model a relationship between multiple predictor variables and a binary/binomial target variable. In case of logistic regression, the linear function is basically used as an input to another function such as in the following relation −

$$h_{\theta(x)} = g(\theta^T x)$$

Here, is the logistic or sigmoid function which can be given as follows −

$$g(z) = \frac{1}{\{1 + e\text{^}\{-z\}\}}$$

To sigmoid curve can be represented with the help of following graph. We can see the values of y-axis lie between 0 and 1 and crosses the axis at 0.5.



The classes can be divided into positive or negative. The output comes under the probability of positive class if it lies between 0 and 1. For our implementation, we are interpreting the output of hypothesis function as positive if it is 0.5, otherwise negative.

Conclusion:

Multiple Linear Regression:

Best for continuous output with linear relationships

Simple and fast, but not suitable for non-linear patterns

Logistic Regression:

Suitable for binary classification

Gives probabilistic output, but limited to two-class problems

LOGISTIC REGRESSION (Iris Dataset)

```
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import numpy as np
import matplotlib.pyplot as plt
dosage = np.array([20, 30, 40, 50])
```

```
cured = np.array([1, 2, 4, 6])
total = np.array([5, 6, 6, 7])

probability = cured / total
print("Probability")
print(probability)

odds = probability / (1 - probability)
print("\nOdds")
print(odds)
log_odds = np.log(odds)
print("\nLog Odds")
print(log_odds)

m, c = np.polyfit(dosage, log_odds, 1)
print("\nRegression Equation")
print("Logit =",round(m,3), "* Dosage +", round(c,3))

new_dosage = 45
logit = m * new_dosage + c
print("\nPredicted Log Odds =", round(logit,3))

prob = np.exp(logit) / (1 + np.exp(logit))
print("Predicted Probability =", round(prob,3))

if prob >= 0.5:
   print("Prediction : CURED")
else:
   print("Prediction : NOT CURED")

x = np.linspace(15,55,100)
y = m*x + c
plt.figure(figsize=(6,4))
plt.scatter(dosage, log_odds, color='red', label='Log Odds')
plt.plot(x,y,label='Regression Line')
plt.xlabel("Dosage")
```

```
plt.ylabel("Log Odds")
plt.title("Regression of Log Odds")
plt.grid(True)
plt.legend()
plt.show()

prob_curve = np.exp(y)/(1+np.exp(y))
plt.figure(figsize=(6,4))
plt.scatter(dosage, probability, color='red', label='Actual Probability')
plt.plot(x,prob_curve,label='Sigmoid Curve')
plt.xlabel("Dosage")
plt.ylabel("Probability of Cure")
plt.title("Logistic Regression Curve")
plt.grid(True)
plt.legend()
plt.show()
```

Output:

Probability [0.2 0.33333333 0.66666667 0.85714286]

Odds [0.25 0.5 2. 6.]

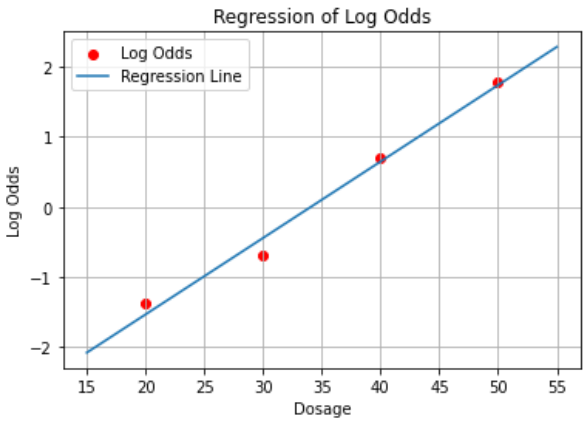
Log Odds [-1.38629436 -0.69314718 0.69314718 1.79175947]

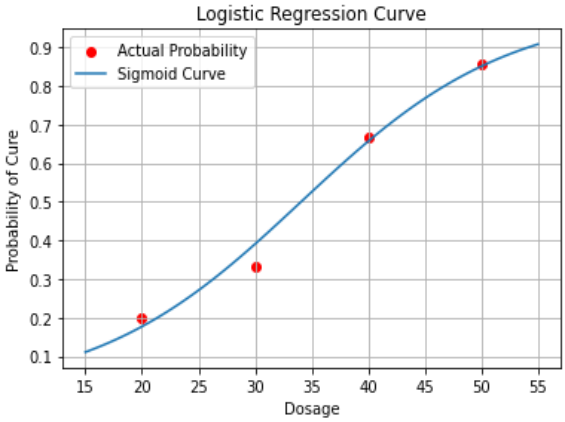
Regression Equation Logit = 0.109 * Dosage + -3.721

Predicted Log Odds = 1.193

Predicted Probability = 0.767

Prediction : CURED





ASSIGNEMENT NO. 4

Support Vector Machine (SVM)

TITLE: Using a suitable dataset, apply **Support Vector Machine (SVM)** to identify the best hyperplane that differentiates the data.

OBJECTIVES:

- To understand the concept and mathematical intuition behind Support Vector Machines (SVM).
- To implement SVM using a suitable dataset for classification.
- To visualize the decision boundary (hyperplane) that separates different classes.
- To evaluate the performance of the SVM model using accuracy, confusion matrix, and classification report.

PROBLEM STATEMENT:

Given a labeled dataset, apply the Support Vector Machine algorithm to classify the data points into appropriate classes by identifying the optimal separating hyperplane. The goal is to maximize the margin between classes and ensure minimum classification error.

SOFTWARE REQUIRED:

- **Python**
- **Libraries: scikit-learn, numpy, matplotlib, pandas**

INPUT: Dataset

OUTPUT:

- A trained SVM model
- Visualization of the decision boundary (hyperplane)
- Model evaluation metrics (accuracy, confusion matrix, precision, recall, F1-score)

THEORY:

Support vector machines (SVMs) are powerful yet flexible supervised machine learning algorithm which is used for both classification and regression. But commonly used in classification problems. SVMs have their unique way of implementation as compared to other machine learning algorithms. The goal of SVM is to find a hyperplane that separates the data points into different classes. A hyperplane is a line in 2D space, a plane in 3D space, or a higher-dimensional surface in n-dimensional space. The hyperplane is chosen in such a way that it maximizes the margin, which is

the distance between the hyperplane and the closest data points of each class. The closest data points are called the support vectors.

The distance between the hyperplane and a data point "x" can be calculated using the formula −

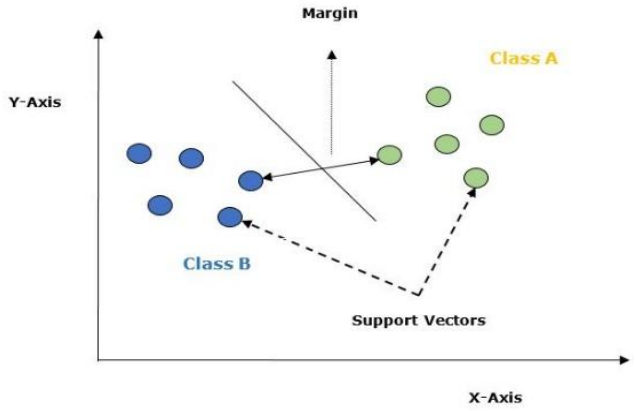
distance = (w . x + b) / ||w||

where "w" is the weight vector, "b" is the bias term, and "||w||" is the Euclidean norm of the weight vector. The weight vector "w" is perpendicular to the hyperplane and determines its orientation, while the bias term "b" determines its position.

The optimal hyperplane is found by solving an optimization problem, which is to maximize the margin subject to the constraint that all data points are correctly classified. In other words, we want to find the hyperplane that maximizes the margin between the two classes while ensuring that no data point is misclassified. This is a convex optimization problem that can be solved using quadratic programming.

If the data points are not linearly separable, we can use a technique called kernel trick to map the data points into a higher-dimensional space where they become separable. The kernel function computes the inner product between the mapped data points without computing the mapping itself. This allows us to work with the data points in the higherdimensional space without incurring the computational cost of mapping them.

Let's understand it in detail with the help of following diagram −



Given below are the important concepts in SVM −

- **Support Vectors** − Datapoints that are closest to the hyperplane is called support vectors. Separating line will be defined with the help of these data points.
- **Hyperplane** − As we can see in the above diagram it is a decision plane or space which is divided between a set of objects having different classes.
- **Margin** − It may be defined as the gap between two lines on the closet data points of different classes. It can be calculated as the perpendicular distance from the line to the support vectors. Large margin is considered as a good margin and small margin is considered as a bad margin.

Conclusion:

In this lab experiment, we successfully implemented the Support Vector Machine (SVM) algorithm and can visualize the optimal hyperplane that separates the two classes.

Sample Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
# Load dataset
iris = datasets.load_iris()
X = iris.data[:, :2]  # using first two features for visualization
y = (iris.target == 0).astype(int)  # Binary classification: Setosa vs Non-Setosa

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Train SVM model
model = SVC(kernel='linear')
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Evaluation
print("Classification Report:\n", classification_report(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

# Visualization
def plot_hyperplane(X, y, model):
    plt.scatter(X[:, 0], X[:, 1], c=y, cmap='bwr', edgecolors='k')
    ax = plt.gca()
    xlim = ax.get_xlim()
    ylim = ax.get_ylim()
```

```
    xx = np.linspace(xlim[0], xlim[1])
    yy = np.linspace(ylim[0], ylim[1])
    YY, XX = np.meshgrid(yy, xx)
    xy = np.vstack([XX.ravel(), YY.ravel()]).T
    Z = model.decision_function(xy).reshape(XX.shape)

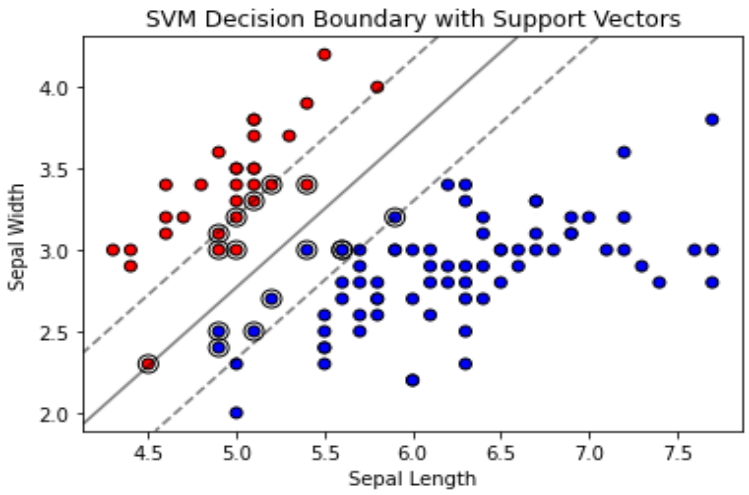
    ax.contour(XX, YY, Z, colors='k', levels=[-1, 0, 1], alpha=0.5,
               linestyles=['--', '-', '--'])
    support_vectors = model.support_vectors_
    ax.scatter(support_vectors[:, 0], support_vectors[:, 1], s=100,
               linewidth=1, facecolors='none', edgecolors='k')
    plt.title('SVM Decision Boundary with Support Vectors')
    plt.xlabel('Sepal Length')
    plt.ylabel('Sepal Width')
    plt.show()
plot_hyperplane(X_train, y_train, model)
```

```
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        26
           1       1.00      1.00      1.00        19

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45

Confusion Matrix:
 [[26  0]
 [ 0 19]]
```



ASSIGNEMENT NO. 5

k-Nearest Neighbors (k-NN)

AIM: Implement k-Nearest Neighbors (k-NN) for:

- Classifying data points into predefined categories.
- Predicting numerical outputs for regression problems.
- Evaluate how the choice of K impacts model performance.

OBJECTIVES:

- To understand the working of the k-Nearest Neighbors (k-NN) algorithm.
- To apply k-NN for classification and regression problems using a suitable dataset.
- To evaluate how varying the value of K (number of neighbors) influences model performance.
- To visualize and interpret the outcomes of the k-NN algorithm.

PROBLEM STATEMENT:

Implement and analyze the **k-NN algorithm** to classify data points into predefined classes and predict continuous values using regression. Investigate the impact of different values of **K** on model accuracy and error rates.

SOFTWARE REQUIRED:

- **Python**
- **Libraries: scikit-learn, numpy, matplotlib, pandas**

INPUT: Dataset

OUTPUT:

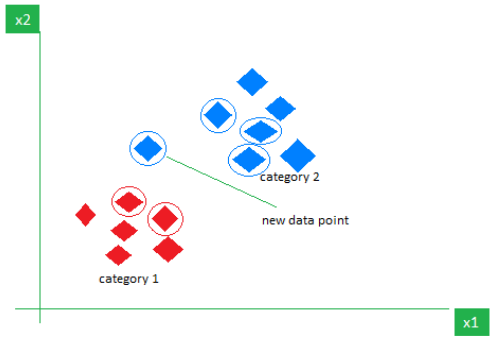
- Predicted class labels for classification and predicted numerical values for regression.
- Accuracy score and confusion matrix for classification.
- Mean Squared Error (MSE) for regression.
- Graphs showing how performance varies with different K values.

THEORY:

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm generally used for classification but can also be used for regression tasks. It works by finding the "k" closest data points (neighbors) to a given input and makes a prediction based on the majority class (for classification) or the average value (for regression). Since KNN makes no assumptions about the underlying data distribution it makes it a non-parametric and instance-based learning method.

K-Nearest Neighbors is also called as a lazy learner algorithm because it does not learn from the training set immediately instead it stores the dataset and at the time of classification it performs an action on the dataset.

For example, consider the following table of data points containing two features:



The new point is classified as Category 2 because most of its closest neighbors are blue squares. KNN assigns the category based on the majority of nearby points. The image shows how KNN predicts the category of a new data point based on its closest neighbours.

- The red diamonds represent Category 1 and the blue squares represent Category 2.
- The new data point checks its closest neighbors (circled points).
- Since the majority of its closest neighbors are blue squares (Category 2) KNN predicts the new data point belongs to Category 2.

KNN works by using proximity and majority voting to make predictions.

'K' in K Nearest Neighbour:

In the k-Nearest Neighbours algorithm k is just a number that tells the algorithm how many nearby points or neighbors to look at when it makes a decision.

Example: Imagine you're deciding which fruit it is based on its shape and size. You compare it to fruits you already know.

- If k = 3, the algorithm looks at the 3 closest fruits to the new one.
- If 2 of those 3 fruits are apples and 1 is a banana, the algorithm says the new fruit is an apple because most of its neighbors are apples.

Selecting value of k for KNN Algorithm:

- The value of k in KNN decides how many neighbors the algorithm looks at when making a prediction.
- Choosing the right k is important for good results.
- If the data has lots of noise or outliers, using a larger k can make the predictions more stable.
- But if k is too large the model may become too simple and miss important patterns and this is called underfitting.
- So k should be picked carefully based on the data.

Conclusion:

- The k-NN algorithm is simple yet powerful for both classification and regression tasks.
- The performance of k-NN heavily depends on the choice of K: Too low a value may lead to overfitting and too high a value may lead to underfitting.
- For classification, model accuracy varies with K and often shows a peak at an optimal value.
- For regression, error (MSE) decreases to a minimum and then may rise again with increasing K.

Sample Code:

Classification Problem:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

iris = load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
print("Accuracy =", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report")
print(classification_report(y_test, y_pred))
```

Regression:

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

housing = fetch_california_housing()
X = housing.data
y = housing.target
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
knn = KNeighborsRegressor(n_neighbors=3)

knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
print(y_pred)
print("RMSE =", np.sqrt(mean_squared_error(y_test, y_pred)))
print("R2 Score =", r2_score(y_test, y_pred))
```

ASSIGNEMENT NO. 6

Hidden Markov Model (HMM)

Aim: Implement a **Hidden Markov Model (HMM)** with both the Forward Algorithm and the Viterbi Algorithm.

OBJECTIVES

1. To understand the working principles of Hidden Markov Models (HMM).
2. To implement the Forward Algorithm for computing the probability of an observation sequence given an HMM.
3. To implement the Viterbi Algorithm for finding the most likely sequence of hidden states for a given observation sequence.
4. To gain practical experience in applying probabilistic models to sequential data.
5. To compare results of probability estimation (Forward) vs. optimal state decoding (Viterbi).

PROBLEM STATEMENT

Sequential data, such as speech, text, weather prediction, or biological sequences, often involve hidden states that cannot be directly observed but influence the observations. The challenge is to:

1. Compute the likelihood of a given observation sequence (Forward Algorithm).
2. Determine the most probable hidden state sequence that could generate the given observations (Viterbi Algorithm).

This lab focuses on implementing both algorithms using a suitable dataset and interpreting the results**.**

SOFTWARE/TOOLS USED

- Programming Language: Python 3.x
- Libraries: NumPy, hmm learn (optional), matplotlib (for visualization)
- IDE: Jupyter Notebook / PyCharm / VS Code

THEORY

Hidden Markov Model (HMM)

An HMM is a statistical model defined by:

- States (S): A finite set of hidden states (e.g., Rainy, Sunny).
- Observations (O): A sequence of symbols generated (e.g., Walk, Shop, Clean).
- Transition Probabilities (A): Probability of moving from one hidden state to another.
- Emission Probabilities (B): Probability of observing a symbol given a state.
- Initial Probabilities (π): Probability distribution of the initial states.

Formally, an HMM is represented as $\lambda = (A, B, \pi)$.

Forward Algorithm

The Forward Algorithm calculates the probability of an observation sequence for a given HMM. It considers all possible hidden-state sequences that could generate those observations.

Example: What is the probability of observing Walk → Shop → Clean, considering all possible combinations of Rainy and Sunny states?

The forward probability, $\alpha_t(j)$ is the probability of observing the sequence up to time t and being in state j at that time.

The Forward algorithm computes the probability of observing a sequence $O = (o_1, o_2, \ldots, o_T)$ given the HMM λ.

- **Step 1:** Initialization

$$\alpha_1(i) = \pi_i \cdot b_i(o_1)$$

- **Step 2:** Induction

$$\alpha_{t+1}(j) = \left(\sum_{i=1}^{N} \alpha_t(i) \cdot a_{ij}\right) \cdot b_j(o_{t+1})$$

- **Step 3:** Termination

$$P(O|\lambda) = \sum_{i=1}^{N} \alpha_T(i)$$

Viterbi Algorithm

The Viterbi Algorithm finds the most probable hidden-state sequence that could generate the given observations.

Example: For Walk → Shop → Clean, which weather sequence is most likely—Sunny → Rainy → Rainy, or another sequence?

The Viterbi value $\delta_t(j)$ is the probability of the most probable path that generates the observations up to time t and ends in state j

The Viterbi algorithm finds the most probable sequence of states $Q = (q_1, q_2, \ldots, q_T)$.

- **Step 1:** Initialization

$$\delta_1(i) = \pi_i \cdot b_i(o_1), \quad \psi_1(i) = 0$$

- **Step 2:** Recursion

$$\delta_t(j) = \max_i \left[\delta_{t-1}(i) \cdot a_{ij}\right] \cdot b_j(o_t)$$

$$\psi_t(j) = \arg\max_i \left[\delta_{t-1}(i) \cdot a_{ij}\right]$$

- **Step 3:** Termination

$$P^* = \max_i \delta_T(i), \quad q_T^* = \arg\max_i \delta_T(i)$$

- **Step 4:** Backtracking

  Trace back ψ to recover the most likely state sequence.

CONCLUSION

- The Forward Algorithm provides the likelihood of observing a given sequence.
- The Viterbi Algorithm provides the most probable hidden state sequence explaining the observations.
- Together, they demonstrate how HMMs are powerful for modeling sequential data where states are hidden (e.g., speech recognition, POS tagging, weather forecasting).

Sample Code

```
import numpy as np
states = ["Rainy", "Sunny"]
observations = ["Walk", "Shop", "Clean"]
obs_seq = [0, 1, 2]   # Walk=0, Shop=1, Clean=2
# HMM Parameters
pi = np.array([0.6, 0.4])   # Initial probabilities
A = np.array([[0.7, 0.3],   # Transition matrix
              [0.4, 0.6]])
B = np.array([[0.1, 0.4, 0.5],   # Emission matrix
              [0.6, 0.3, 0.1]])
# ------------------ Forward Algorithm ------------------
def forward(obs_seq, A, B, pi):
    N = len(A)  # number of states
    T = len(obs_seq)  # length of observation sequence
    alpha = np.zeros((T, N))
    # Initialization
    alpha[0] = pi * B[:, obs_seq[0]]
```

```
    # Induction
    for t in range(1, T):
        for j in range(N):
            alpha[t, j] = np.sum(alpha[t-1] * A[:, j]) * B[j, obs_seq[t]]

    # Termination
    return np.sum(alpha[T-1])
# ------------------ Viterbi Algorithm ------------------
def viterbi(obs_seq, A, B, pi):
    N = len(A)
    T = len(obs_seq)

    delta = np.zeros((T, N))
    psi = np.zeros((T, N), dtype=int)

    # Initialization
    delta[0] = pi * B[:, obs_seq[0]]

    # Recursion
    for t in range(1, T):
        for j in range(N):
            seq_probs = delta[t-1] * A[:, j]
            psi[t, j] = np.argmax(seq_probs)
            delta[t, j] = np.max(seq_probs) * B[j, obs_seq[t]]

    # Termination
    best_path_prob = np.max(delta[T-1])
    best_last_state = np.argmax(delta[T-1])

    # Backtracking
    best_path = [best_last_state]
    for t in range(T-1, 0, -1):
        best_last_state = psi[t, best_last_state]
        best_path.insert(0, best_last_state)
    return best_path_prob, [states[i] for i in best_path]
# ------------------ Run ------------------
forward_prob = forward(obs_seq, A, B, pi)
viterbi_prob, viterbi_path = viterbi(obs_seq, A, B, pi)
```

```
print("Forward Algorithm: Probability of Observations =", forward_prob)
print("Viterbi Algorithm: Best Path Probability =", viterbi_prob)
print("Most Likely State Sequence =", viterbi_path)
```

Expected Input: Observation sequence: [Walk, Shop, Clean]

Probable Output

Forward Algorithm: Probability of Observations = 0.033612

Viterbi Algorithm: Best Path Probability = 0.01344

Most Likely State Sequence = ['Sunny', 'Rainy', 'Rainy']

ASSIGNEMENT NO. 7

Clustering Algorithms

AIM: Using public domain datasets in UCI ML repository, Implementation of clustering algorithms (K-Means and K-Medoid)

OBJECTIVES

- To understand the concept and importance of clustering in unsupervised machine learning.
- To implement clustering algorithms such as K-Means and K-Medoids on real-world datasets.
- To analyze and compare clustering results using different evaluation parameters.
- To visualize the data clusters and interpret group patterns.
- To gain practical experience in applying clustering for pattern recognition and data segmentation.

PROBLEM STATEMENT

Given a dataset from the UCI Machine Learning Repository, implement different clustering techniques to group data instances into clusters based on their similarities. The task is to observe how different algorithms (K-Means and K-Medoids) partition the data and to interpret the resulting clusters in a meaningful way

THEORY

Clustering is an unsupervised learning technique that groups similar data points into clusters such that data within a cluster are more similar to each other than to those in other clusters.

a) K-Means Clustering

K-Means aims to partition data into *k* clusters by minimizing the intra-cluster variance (sum of squared distances between data points and their respective cluster centroids).

Algorithm Steps:

1. Select *k* initial centroids randomly.
2. Assign each data point to the nearest centroid.
3. Recalculate centroids as the mean of all points in each cluster.
4. Repeat steps 2–3 until centroids do not change significantly.

Advantages: Simple, efficient, and scalable.

Disadvantages: Sensitive to outliers and requires specifying *k* in advance.

b) K-Medoids Clustering

K-Medoids (or PAM – Partitioning Around Medoids) is similar to K-Means but uses actual data

points (medoids) instead of mean values as cluster centers.

Algorithm Steps:

1. Select *k* data points as initial medoids.
2. Assign each point to the nearest medoid.
3. For each cluster, choose a new medoid that minimizes the total dissimilarity.
4. Repeat until medoids stabilize.

Advantages: More robust to noise and outliers than K-Means.

Disadvantages: Computationally expensive for large datasets.

SOFTWARE/TOOLS USED

Python 3.x	**Programming language**
NumPy	**Numerical computations**
Pandas	**Data manipulation and analysis**
Matplotlib / Seaborn	**Data visualization**
Scikit-learn	**Implementation of K-Means**
PyCaret / scikit-learn-extra	**Implementation of K-Medoids**
UCI ML Repository	**Source of datasets**

CONCLUSION

- Clustering is an essential unsupervised learning method for discovering inherent patterns in data.
- **K-Means** is suitable for large, well-separated datasets, while **K-Medoids** performs better when outliers are present.
- The implementation demonstrates how clustering can uncover structure in real-world datasets, enabling insights without labeled data.

Sample Code

K-Means Clustering

```
# Import libraries
import pandas as pd
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
```

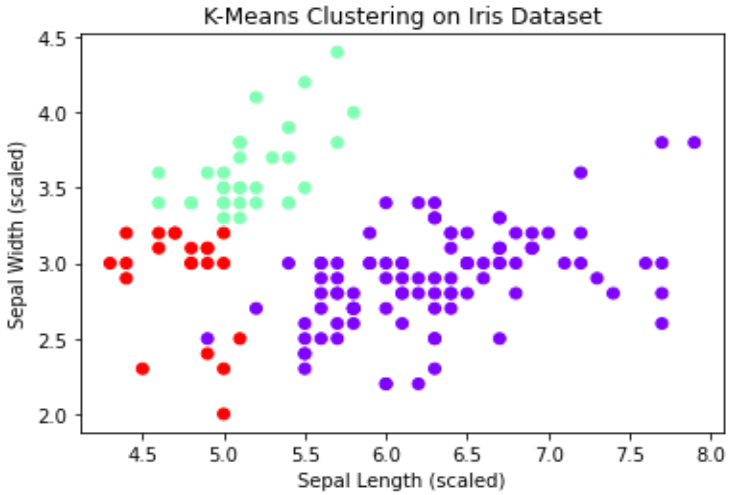
```
from sklearn.datasets import load_iris

# Load dataset
iris = load_iris()
data = pd.DataFrame(iris.data, columns=iris.feature_names)

# Standardize data
scaler = StandardScaler()
scaled_data = scaler.fit_transform(data)

kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(scaled_data)
data['Cluster'] = kmeans.labels_

plt.scatter(data.iloc[:, 0], data.iloc[:, 1], c=data['Cluster'], cmap='rainbow')
plt.title("K-Means Clustering on Iris Dataset")
plt.xlabel("Sepal Length (scaled)")
plt.ylabel("Sepal Width (scaled)")
plt.show()
```



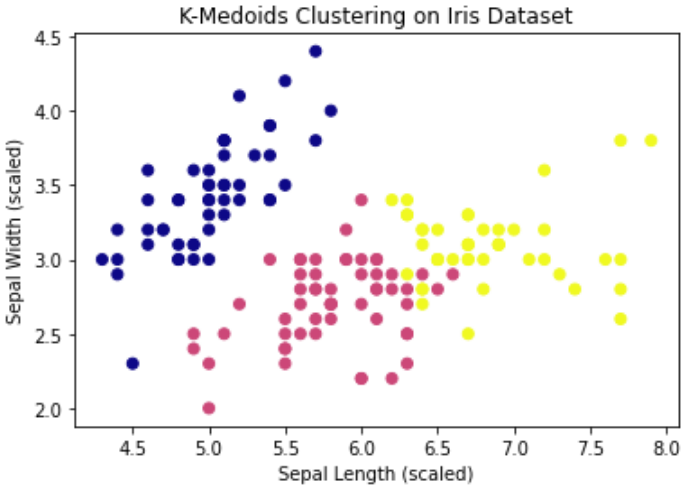
Sample Code – K-Medoids Clustering

```
# Import necessary libraries
from sklearn_extra.cluster import KMedoids

# Apply K-Medoids
kmedoids = KMedoids(n_clusters=3, random_state=42)
kmedoids.fit(scaled_data)
```

```
data['KMedoid_Cluster'] = kmedoids.labels_

# Visualization
plt.scatter(data.iloc[:, 0], data.iloc[:, 1], c=data['KMedoid_Cluster'], cmap='plasma')
plt.title("K-Medoids Clustering on Iris Dataset")
plt.xlabel("Sepal Length (scaled)")
plt.ylabel("Sepal Width (scaled)")
plt.show()
```



ASSIGNEMENT NO. 8

DBSCAN algorithm

AIM: Implement the DBSCAN clustering algorithm using a suitable dataset. Identify clusters and noise points, visualize the results, and comment on the clustering obtained.

OBJECTIVES

- To understand the fundamentals of density-based clustering.
- To implement DBSCAN in Python using scikit-learn.
- To understand the effect of **eps** and **min_samples**.
- To identify core points, border points, and noise points.
- To visualize and interpret the resulting clusters.

PROBLEM STATEMENT

Develop a Python program to group two-dimensional data points using DBSCAN. Select suitable values of eps and min_samples, identify clusters and noise points, and analyze the results.

SOFTWARES USED

Python

Libraries: scikit-learn, numpy, matplotlib

INPUT

A dataset containing one or two-dimensional points.

OUTPUT

- Cluster labels assigned to each point.
- Number of clusters formed.
- Number of noise points.
- A scatter plot showing clusters and noise.
- Observations about the clustering results.

THEORY

Clustering

Clustering is an **unsupervised learning technique** that groups similar data points together. Points within the same cluster are more similar to one another than to points in other clusters.

DBSCAN

DBSCAN stands for **Density-Based Spatial Clustering of Applications with Noise**.

It forms clusters by connecting points in dense regions. Points that do not belong to any cluster are identified as **noise**.

For example, consider locations of houses on a map. Houses located close together can form a cluster, while an isolated house may be identified as noise.

DBSCAN does not require the number of clusters to be specified in advance and can identify clusters with irregular shapes.

Important Parameters

1. eps (ε):

The maximum distance used to decide whether two points are neighbours.

- A small value may produce more noise points.
- A large value may combine separate groups into one cluster.

2. min_samples:

The minimum number of points required within the eps neighbourhood for a point to become a core point. The count includes the point itself.

Types of Points

Point type	Meaning
Core point	Has at least min_samples points, including itself, within its eps neighbourhood.
Border point	Has fewer than min_samples neighbours but lies within the eps neighbourhood of a core point.
Noise point	Is neither a core point nor a border point.

Working of DBSCAN

1. Select an unvisited point and find its neighbours within distance **eps**.
2. If it has at least **min_samples** points in its neighbourhood, start a new cluster.
3. Expand the cluster through neighbouring core points and include their border points.
4. If the selected point is not a core point, mark it as noise temporarily. It may later become a border point.
5. Repeat until all points have been visited.

CONCLUSION

DBSCAN successfully groups the dataset into **two clusters and one noise point**. It is useful for identifying dense groups, irregularly shaped clusters, and outliers. However, parameter selection is important, and datasets with widely varying densities can be difficult to cluster.

Sample Code

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN

X = np.array([
    [1, 1],   # A
    [2, 1],   # B
    [3, 1],   # C
    [7, 5],   # D
    [8, 5],   # E
    [9, 5],   # F
    [12, 9]   # G
])

names = [f"P{i+1}" for i in range(len(X))]
# Names used in the given example
names = list("ABCDEFG")

# Apply DBSCAN
model = DBSCAN(eps=1, min_samples=3, metric="euclidean")
labels = model.fit_predict(X)

# Identify core points
core_indices = set(model.core_sample_indices_)

# Display cluster and point type
print("Point  Cluster  Type")
```

```
for i, label in enumerate(labels):
    if label == -1:
        cluster = "Noise"
        point_type = "Noise"
    else:
        cluster = str(label + 1)
        point_type = "Core" if i in core_indices else "Border"

    print(f"{names[i]:<6} {cluster:<8} {point_type}")

# Plot clusters
for label in sorted(set(labels)):
    points = X[labels == label]

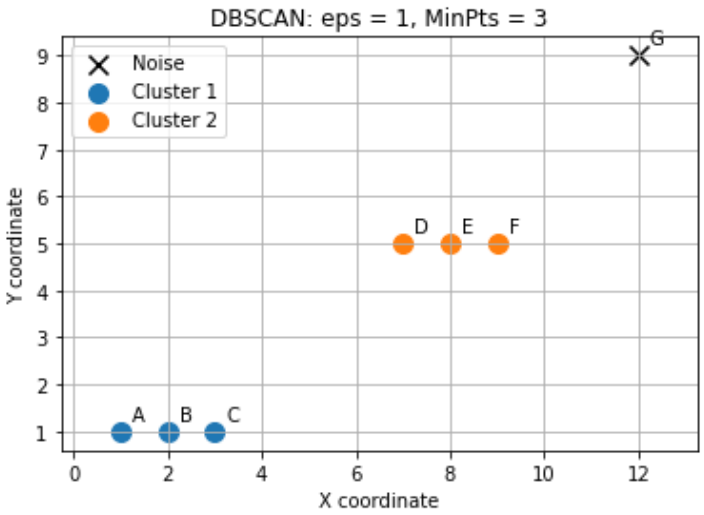
    if label == -1:
        plt.scatter(
            points[:, 0], points[:, 1],
            color="black", marker="x", s=100, label="Noise"
        )
    else:
        plt.scatter(
            points[:, 0], points[:, 1],
            s=100, label=f"Cluster {label + 1}"
        )

# Add point names
for i, (x, y) in enumerate(X):
    plt.annotate(names[i], (x, y), xytext=(6, 6),
                 textcoords="offset points")

plt.xlabel("X coordinate")
plt.ylabel("Y coordinate")
plt.title("DBSCAN: eps = 1, MinPts = 3")
plt.legend()
plt.grid(True)
plt.axis("equal")
```

```
plt.show()
```

Point	Cluster	Type
A	1	Border
B	1	Core
C	1	Border
D	2	Border
E	2	Core
F	2	Border
G	Noise	Noise



ASSIGNEMENT NO. 9

AdaBoost algorithm

AIM: Implement an ensemble algorithm (Ada Boost or Gradient Boosting) and discuss its performance on a given dataset.

OBJECTIVES

- To understand the concept and working of ensemble learning methods.
- To implement an ensemble algorithm (AdaBoost or Gradient Boosting) using Python.
- To evaluate the performance of the model using standard performance metrics.

PROBLEM STATEMENT

Develop a Python program to implement AdaBoost on the Iris dataset. Evaluate its classification performance and compare the results with a single decision tree..

SOFTWARE REQUIRED

Programming Language: Python

Libraries:

- pandas — data handling
- numpy — numerical computation
- sklearn — model implementation and evaluation
- matplotlib / seaborn — visualization

INPUT

The Iris dataset, containing 150 flower samples with four features

OUTPUT

- Trained AdaBoost classification model.
- Predicted flower classes.
- Accuracy, precision, recall, and F1-score.
- Confusion matrix.
- Performance comparison with a single decision tree.

THEORY

Ensemble Learning:

Ensemble learning combines predictions from multiple models to obtain a final prediction. For example, several classifiers may predict the type of a flower. Their predictions are combined to make the final classification. Common ensemble techniques include **bagging, boosting, and stacking**.

- Bagging (Bootstrap Aggregating): Reduces variance (e.g., Random Forest).
- Boosting: Reduces bias by sequentially improving weak learners.

Boosting

Boosting trains models sequentially. Each new model attempts to improve upon the errors made by the previous models.

The individual models are usually simple models called **weak learners**. Combining them produces a stronger model.

AdaBoost

AdaBoost stands for **Adaptive Boosting**.

It initially assigns equal weights to all training samples. After each weak learner is trained:

- Incorrectly classified samples receive greater importance.
- Correctly classified samples receive less importance.
- The next learner focuses more on the difficult samples.

The final prediction combines the learners using a **weighted vote**, where better-performing learners receive greater voting importance.

A commonly used weak learner is a **decision stump**, which is a decision tree with only one split.

Important Parameters:

Parameter	Meaning
n_estimators	Maximum number of weak learners to train.
learning_rate	Controls the contribution of each learner.
estimator	The base model used as the weak learner.
random_state	Helps reproduce the same results.

Algorithm Steps:

1. Initialize equal weights for all samples.
2. Train a weak classifier.
3. Calculate the error rate.

4. Update weights — increase for misclassified samples.
5. Combine weak learners into a strong classifier.

CONCLUSION

- The ensemble model (AdaBoost) achieved higher accuracy compared to a single Decision Tree.
- Boosting effectively reduces bias and improves generalization by focusing on difficult samples.
- Hence, ensemble algorithms like AdaBoost are powerful techniques for both classification and regression tasks.

Sample Code

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris
data = load_iris()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42, stratify=y)

base_estimator = DecisionTreeClassifier(max_depth=1)
model = AdaBoostClassifier(
   estimator=base_estimator,
   n_estimators=100,
   learning_rate=0.8,
   random_state=42
)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
print("\nClassification Report:\n")
print(classification_report(
    y_test, y_pred, target_names=data.target_names))

plt.figure(figsize=(8, 4))
plt.barh(X.columns, model.feature_importances_)
plt.title("Feature Importance (AdaBoost - Iris)")
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.tight_layout()
plt.show()
```

Output:

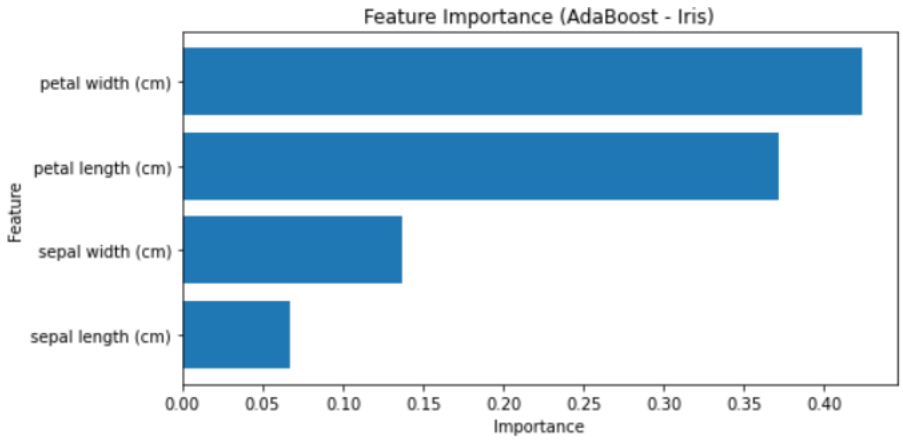
```
Accuracy: 0.9333333333333333

Classification Report:

              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.90      0.90      0.90        10
   virginica       0.90      0.90      0.90        10

    accuracy                           0.93        30
   macro avg       0.93      0.93      0.93        30
weighted avg       0.93      0.93      0.93        30
```



ASSIGNEMENT NO. 10

Mini Project

Execute a mini project applying machine learning algorithms to real-world data, covering preprocessing, feature engineering, model building, evaluation, and advanced techniques like ensemble methods or reinforcement learning.

Students will execute a mini project by applying Machine Learning algorithms to real-world datasets. The project will cover the complete ML workflow:

Data Preprocessing: Handling missing values, normalization, encoding, and cleaning.

Feature Engineering: Deriving meaningful features, dimensionality reduction, and feature selection.

Model Building: Implementing suitable algorithms for classification, regression, or clustering.

Model Evaluation: Using relevant metrics such as accuracy, precision, recall, F1-score, AUC, etc.

Advanced Techniques: Exploring methods like ensemble learning, hyperparameter tuning, or reinforcement learning to improve performance.

Group Work: Students will work in teams/individual to plan, develop, and present their project, promoting collaboration, ethical data usage, and basic project management skills

Deployment: Deploy project to any platform like GitHub.